Exercise 1: Variables and Data Types

```
  #Create variables of different types and print them with their types"""
   name = input("Enter name: ")
   age = int(input("Enter age: "))
   height = float(input("Enter height in meters: "))

   print(f"Name: {name} (type: {type(name)})")
   print(f"Age: {age} (type: {type(age)})")
   print(f"Height: {height} (type: {type(height)})")
```

Exercise 2: String Operations

```
   sentence = input("Enter a sentence: ")
   # Convert to uppercase
   uppercase = sentence.upper()
   print(f"Uppercase: {uppercase}")

   # Count vowels
   vowels = "aeiouAEIOU"
   vowel_count = sum(1 for char in sentence if char in vowels)
   print(f"Vowels: {vowel_count}")

   # Reverse the string
   reversed_str = sentence[::-1]
   print(f"Reversed: {reversed_str}")
```

Exercise 3: Lists and List Methods

```
"""Perform list operations"""

numbers = [10, 20, 30, 40, 50]

print(f"Original list: {numbers}")


# Add 60 at the end

numbers.append(60)

print(f"After append(60): {numbers}")


# Insert 15 at index 1

numbers.insert(1, 15)

print(f"After insert(1, 15): {numbers}")


# Remove 30

numbers.remove(30)

print(f"After remove(30): {numbers}")


print(f"Final length: {len(numbers)}")
```

Exercise 4: Tuples

```
"""Find max, min, and average of tuple"""

marks = (85, 92, 78, 95, 88)


maximum = max(marks)

minimum = min(marks)

average = sum(marks) / len(marks)


print(f"Marks: {marks}")

print(f"Maximum: {maximum}")

print(f"Minimum: {minimum}")

print(f"Average: {average:.2f}")
```

```
# Exercise 5: Dictionaries
    """Dictionary operations"""
    students = {'Ram': 85, 'Sita': 90}
    print(f"Initial dictionary: {students}")

    # Add new entry
    students['Lakshman'] = 88
    print(f"After adding Lakshman: {students}")

    # Update Ram's marks
    students['Ram'] = 87
    print(f"After updating Ram's marks: {students}")

    # Display all students
    print("\nAll students with marks:")
    for name, marks in students.items():
        print(f"{name}: {marks}")
```

```
# Exercise 6: Loops - For Loop
    """Print multiplication table"""
    print(f"\nMultiplication table of {num}:")
    for i in range(1, 11):
        print(f"{num} x {i} = {num * i}")
```

```
# Exercise 7: Loops - While Loop
    """Calculate factorial using while loop"""
    if n < 0:
        return "Factorial not defined for negative numbers"
    factorial = 1
    i = 1
    while i <= n:
        factorial *= i
```

```
    i += 1

return factorial
```

```
# Exercise 8: Functions with Multiple Parameters
    """Calculate Simple Interest = (P * R * T) / 100"""
    interest = (principal * rate * time) / 100
    return interest


# Exercise 9: Lambda Functions
    """Create and test lambda functions"""
    add = lambda x, y: x + y
    subtract = lambda x, y: x - y
    multiply = lambda x, y: x * y
    divide = lambda x, y: x / y if y != 0 else "Cannot divide by zero"

    print("Testing lambda functions with 10 and 5:")
    print(f"Addition: {add(10, 5)}")
    print(f"Subtraction: {subtract(10, 5)}")
    print(f"Multiplication: {multiply(10, 5)}")
    print(f"Division: {divide(10, 5)}")
# Exercise 10: List Comprehension
    """Create list of squares of even numbers from 1 to 20"""
    squares = [x**2 for x in range(1, 21) if x % 2 == 0]
    print(f"Squares of even numbers (1-20): {squares}")
    return squares
==================================
# SECTION 2: EXCEPTION HANDLING

# Exercise 11: Basic Try-Except
    """Handle division by zero error"""
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = num1 / num2
        print(f"Result: {result}")
```

```
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")


# Exercise 12: Multiple Exception Handling
    """Handle both ValueError and ZeroDivisionError"""
    try:
        num1 = int(input("Enter first number: "))
        num2 = int(input("Enter second number: "))
        result = num1 / num2
        print(f"Result: {result}")
    except ValueError:
        print("Error: Please enter valid integers!")
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")


# Exercise 13: Try-Except-Else
    """Demonstrate else clause in exception handling"""
    filename = input("Enter filename to read: ")
    try:
        file = open(filename, 'r')
        content = file.read()
        print(content)
        file.close()
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found!")
    else:
```

```
print("File read successfully!")
```

```
# Exercise 14: Try-Except-Finally
    """Demonstrate finally clause"""
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = num1 / num2
        print(f"Result: {result}")
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
    except ValueError:
        print("Error: Please enter valid numbers!")
    finally:
        print("Execution completed")
```

```
# Exercise 15: Raising Exceptions
    """Raise ValueError if num is negative"""
    if num < 0:
        raise ValueError("Negative numbers not allowed!")
    else:
        print(f"{num} is a positive number")
        return num
```

```
# Exercise 16: Custom Exception
    """Custom exception for invalid age"""

    """Raise InvalidAgeError if age is invalid"""
    if age < 0 or age > 120:
```

```
        raise InvalidAgeError(f"Invalid age: {age}. Age must be between 0 and 120.")
    else:
        print(f"Valid age: {age}")
        return age

# Exercise 17: Exception Handling with Lists
    """Handle IndexError when accessing list elements"""
    numbers = [10, 20, 30]
    try:
        print(f"List: {numbers}")
        value = numbers[5]
        print(f"Value at index 5: {value}")
    except IndexError:
        print("Error: Index out of range! List has only 3 elements.")

# Exercise 18: Exception Handling with Dictionaries
    """Handle KeyError when accessing dictionary"""
    student = {'name': 'Ram', 'age': 17}
    try:
        print(f"Dictionary: {student}")
        marks = student['marks']
        print(f"Marks: {marks}")
    except KeyError:
        print("Error: Key 'marks' not found in dictionary!")


# Exercise 19: File Handling with Exception
    """Read a file with complete exception handling"""
    file = None
    try:
        file = open(filename, 'r')
        content = file.read()
```

```
        print(f"File content:\n{content}")
        return content
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found!")
        return None
    except PermissionError:
        print(f"Error: Permission denied to read '{filename}'!")
        return None
    finally:
        if file:
            file.close()
            print("File closed successfully")

# Exercise 20: Comprehensive Exception Handling
    """Create calculator with exception handling"""
    print("Simple Calculator (Enter 'quit' to exit)")

    while True:
        try:
            choice = input("\nEnter operation (+, -, *, /) or 'quit': ")

            if choice.lower() == 'quit':
                print("Goodbye!")
                break

            if choice not in ['+', '-', '*', '/']:
                print("Error: Invalid operation! Use +, -, *, or /")
                continue

            num1 = float(input("Enter first number: "))
            num2 = float(input("Enter second number: "))

            if choice == '+':
```

```
            result = num1 + num2
        elif choice == '-':
            result = num1 - num2
        elif choice == '*':
            result = num1 * num2
        elif choice == '/':
            result = num1 / num2

        print(f"Result: {num1} {choice} {num2} =
{result}")

    except ValueError:
        print("Error: Please enter valid numbers!")
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
```

```
==================================

# BONUS EXERCISES

==================================

# Exercise 21: Nested Try-Except

    """Create nested try-except blocks"""

    try:

        num1 = int(input("Enter first number: "))

        num2 = int(input("Enter second number: "))


        try:

            result = num1 / num2

            print(f"Result: {result}")

        except ZeroDivisionError:

            print("Inner Exception: Cannot divide by zero!")


    except ValueError:

        print("Outer Exception: Please enter valid integers!")



# Exercise 22: Exception with File and List Operations

    """Read numbers from file and calculate sum"""

    try:

        file = open(filename, 'r')

        content = file.read()

        file.close()

        numbers = [int(num.strip()) for num in content.split(',')]

        total = sum(numbers)

        print(f"Numbers: {numbers}")

        print(f"Sum: {total}")

        return total
```

```
except  FileNotFoundError:
    print(f"Error: File '{filename}' not found!")
    return 0
except  ValueError:
    print("Error: File contains non-numeric data!")
    return 0
```